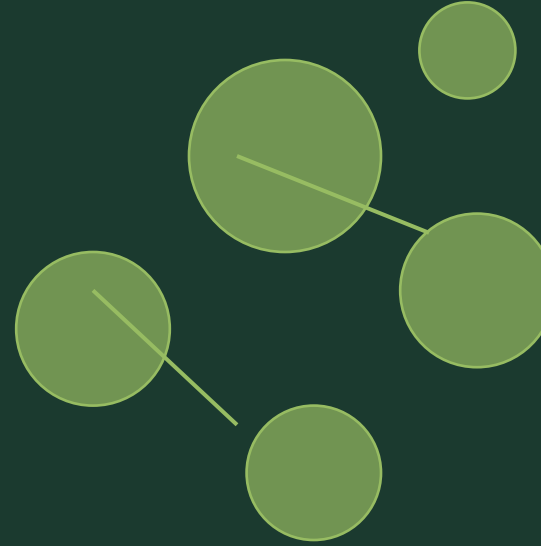




ÁRBOLES AVL

Estructura, balance y rotaciones paso a paso

Estructura de Datos



Contenido de la sesión



1. Origen y motivación

¿Por qué necesitamos árboles auto-balanceados?



3. Factor de balance (FB)

Cómo calcularlo y leerlo correctamente



5. Inserción paso a paso

Desde un nodo hasta detectar el desbalance



7. Complejidad y comparación

AVL vs. BST vs. árbol rojo-negro



2. Definición formal de un AVL

Propiedad del balance, altura y FB



4. Tipos de rotaciones

Simple (LL, RR) y doble (LR, RL)



6. Eliminación y rebalanceo

Casos especiales y cascadas de rotación



8. Pseudocódigo y ejercicios

Implementación y práctica guiada

Origen y motivación

¿Por qué un BST común no siempre nos sirve?

Un poco de historia

Propuestos en 1962 por los matemáticos soviéticos **G. M. Adelson-Velsky** y **E. M. Landis**. Sus iniciales dan nombre a la estructura: **A-V-L**.

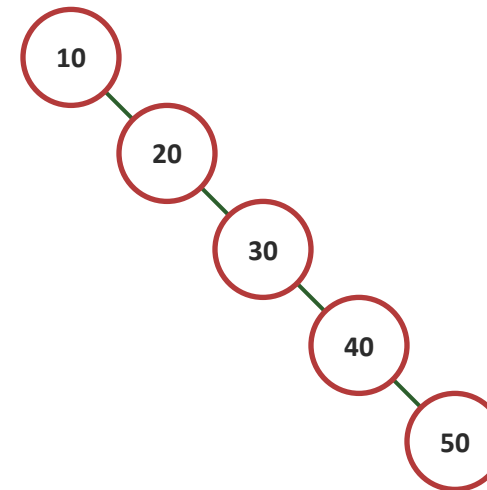
¿Qué problema resuelven?

Un BST puede degenerar en una **lista enlazada** si los datos se insertan ordenados. La búsqueda pasa de $O(\log n)$ a $O(n)$.

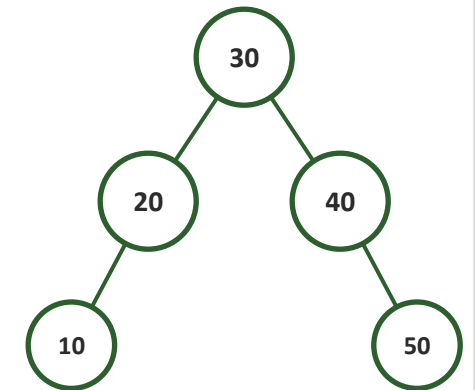
El **AVL** **garantiza altura logarítmica** realizando *rotaciones automáticas* cuando detecta un desbalance.

BST degenerado vs. AVL balanceado

Inserción: 10, 20, 30, 40, 50



BST: altura = 5 $\rightarrow O(n)$



AVL: altura = 3 $\rightarrow O(\log n)$



Mismos datos, dos estructuras: la diferencia está en mantener el equilibrio.

Definición formal

¿Qué hace que un BST sea un AVL?

Definición

Un **árbol AVL** es un *árbol binario de búsqueda* en el cual, para cada nodo, la diferencia entre las alturas de su subárbol izquierdo y derecho es **como máximo 1**.



Altura $h(v)$

Número de aristas en el camino más largo de v a una hoja descendiente.

$$h(\text{hoja}) = 0 \cdot h(\text{null}) = -1$$



Factor de balance

$FB(v) = h(\text{izq}) - h(\text{der})$. Define el desequilibrio de cada nodo.

$$\text{Válido si } FB \in \{-1, 0, +1\}$$



Auto-balanceo

Si tras insertar o eliminar $|FB| > 1$, se aplican rotaciones para restaurar la propiedad.

$$\text{Costo: } O(\log n)$$

Factor de balance (FB)

Cómo se calcula y cómo interpretarlo



Fórmula

$$FB(v) = altura(izq) - altura(der)$$

Lectura de los valores:

FB = 0 → subárboles de igual altura.

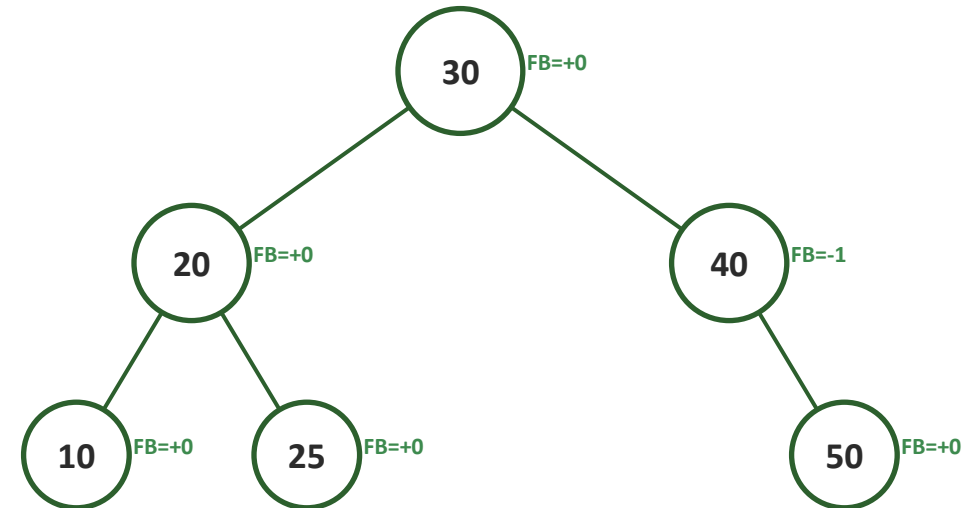
FB = +1 → el izquierdo es más alto por 1.

FB = -1 → el derecho es más alto por 1.

|FB| > 1 → desbalanceado. Hay que rotar.

Nota: algunos textos usan $h(der) - h(izq)$; el signo se invierte pero las reglas son equivalentes.

Ejemplo: ¿está balanceado?



Todos los $FB \in \{-1, 0, +1\}$ → es un AVL válido

Si en algún nodo $|FB| > 1$, hay que rebalancear con una rotación.

Propiedades y operaciones

Lo que un AVL hereda del BST y lo que añade

Operación	BST común	Árbol AVL	Comentario
Búsqueda	$O(n)$ en el peor caso	$O(\log n)$ garantizado	Aprovecha el balance
Inserción	$O(n)$ peor caso	$O(\log n)$ + rotaciones	1 rotación basta
Eliminación	$O(n)$ peor caso	$O(\log n)$ + rotaciones	Puede cascada hasta la raíz
Espacio extra	Solo punteros	+ altura/FB por nodo	1 entero por nodo
Recorridos	In, pre, post $O(n)$	Iguales	El balance no los altera



Ventaja clave

Todas las operaciones críticas se garantizan en tiempo logarítmico.



Costo a considerar

Mantener la altura y rotar consume tiempo en cada inserción/eliminación.

Los 4 tipos de desbalance

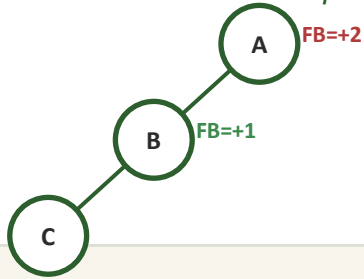
Cada caso requiere un tipo específico de rotación

C A S O

LL

Izquierda – Izquierda

Rotación simple a la derecha

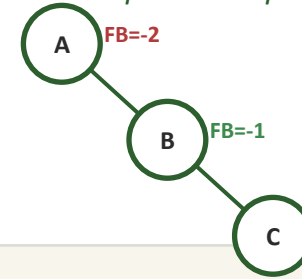


C A S O

RR

Derecha – Derecha

Rotación simple a la izquierda

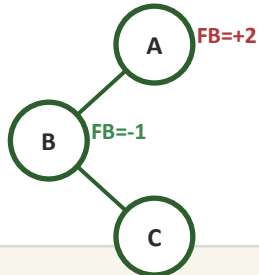


C A S O

LR

Izquierda – Derecha

Rotación doble izq-der

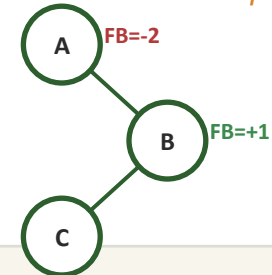


C A S O

RL

Derecha – Izquierda

Rotación doble der-izq

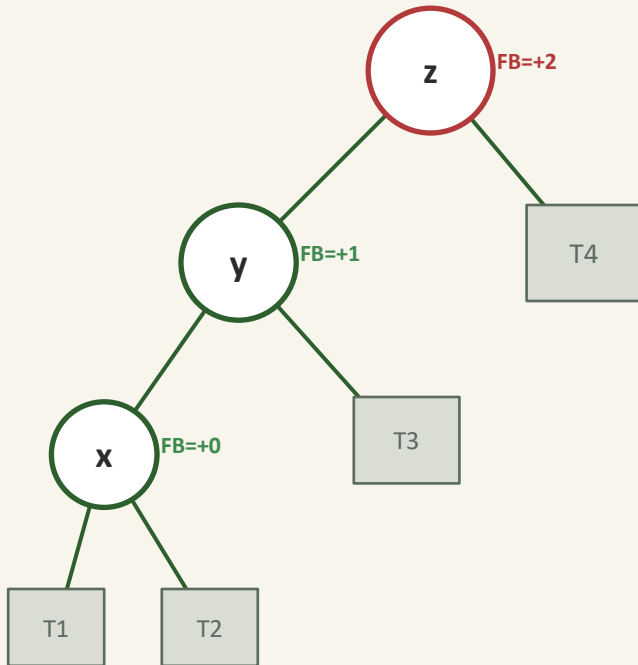


Rotación simple a la derecha (caso LL)

Cuando insertamos en el subárbol izquierdo del hijo izquierdo

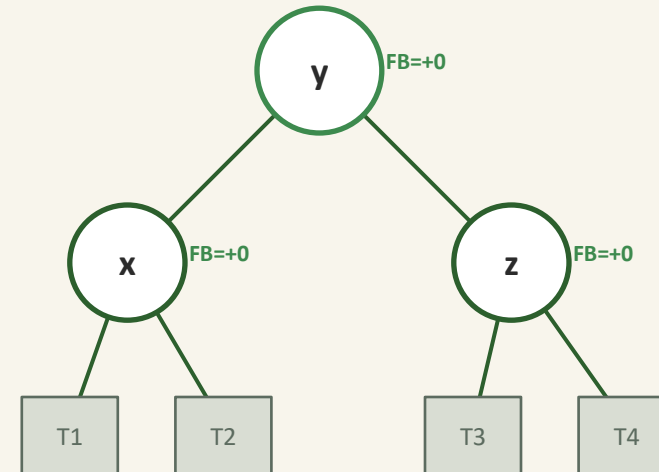
CASO LL

ANTES — z desbalanceado



Rotar a la derecha

DESPUÉS — y nuevo padre



rotarDerecha(z):

$y \leftarrow z.\text{izq}$; $T3 \leftarrow y.\text{der}$

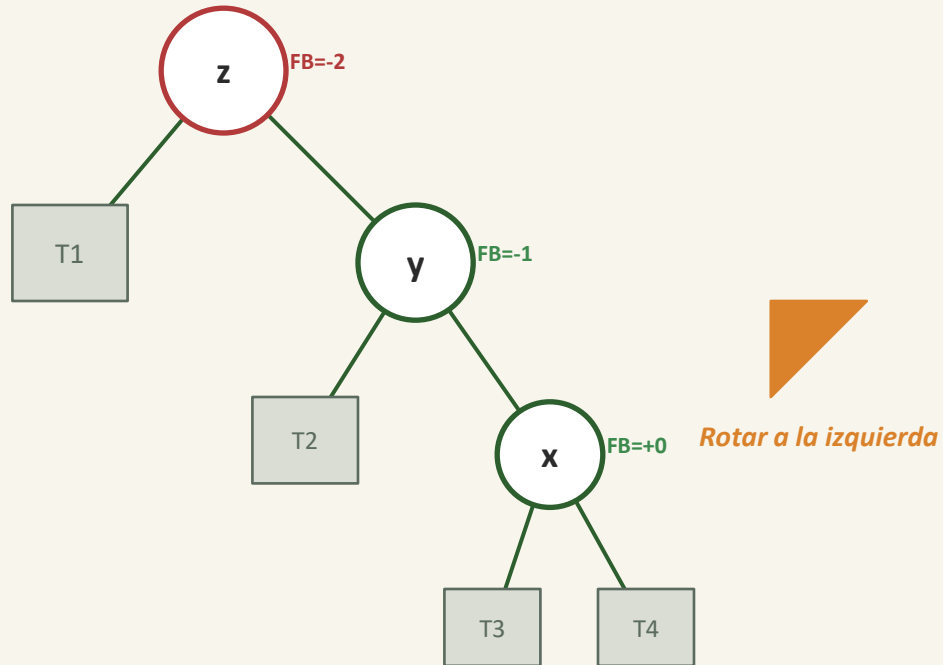
$y.\text{der} \leftarrow z$; $z.\text{izq} \leftarrow T3$; actualizar alturas(z, y) $\Rightarrow$ retornar y como nueva raíz

Rotación simple a la izquierda (caso RR)

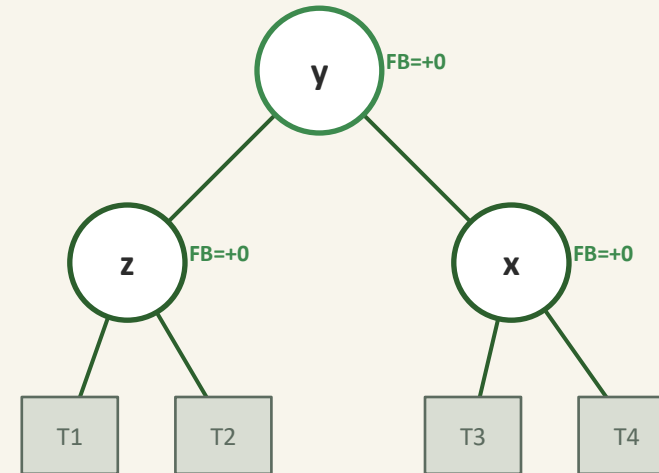
Cuando insertamos en el subárbol derecho del hijo derecho

CASO RR

ANTES — z desbalanceado



DESPUÉS — y nuevo padre



rotarIzquierda(z):

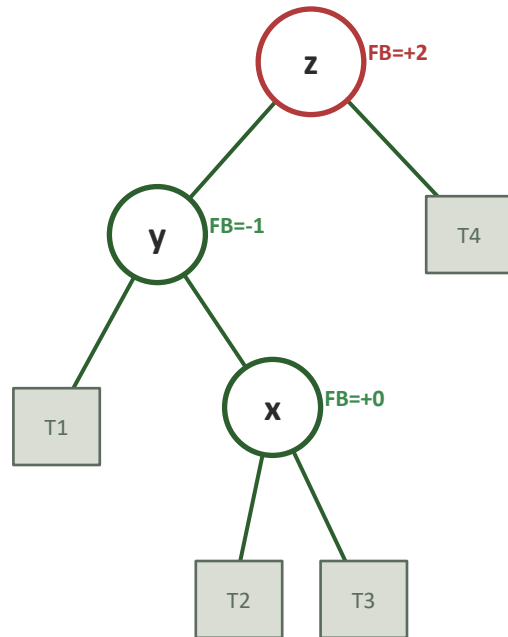
```
y ← z.der ; T2 ← y.izq  
y.izq ← z ; z.der ← T2 ; actualizar alturas(z, y) ⇒ retornar y como nueva raíz
```

Rotación doble Izquierda-Derecha (caso LR)

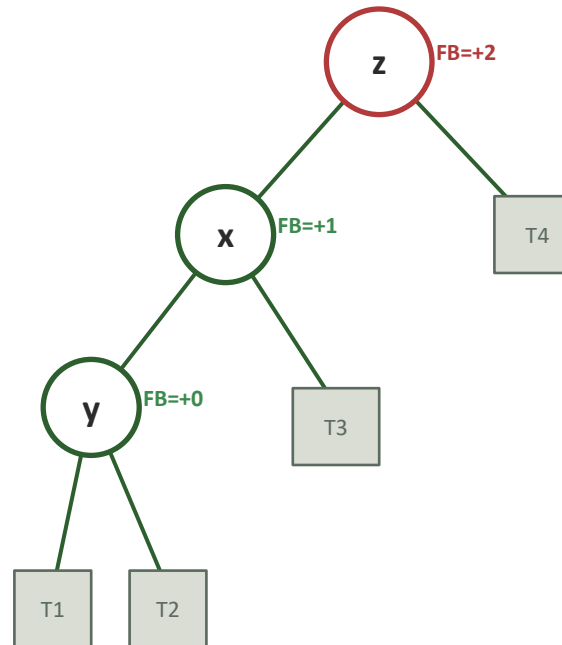
Se aplica una rotación izquierda al hijo, después una rotación derecha al nodo

CASO LR

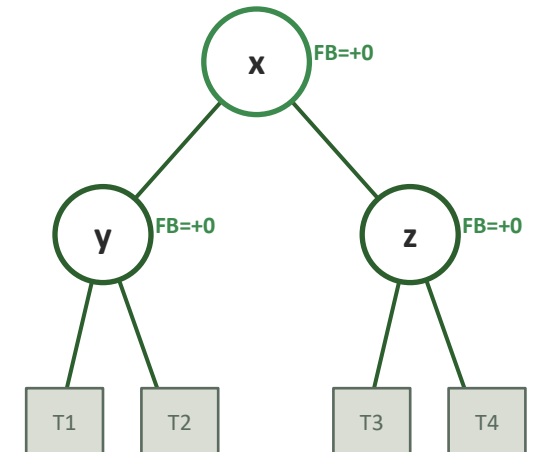
1) ANTES



2) Rotar y a la izquierda



3) Rotar z a la derecha

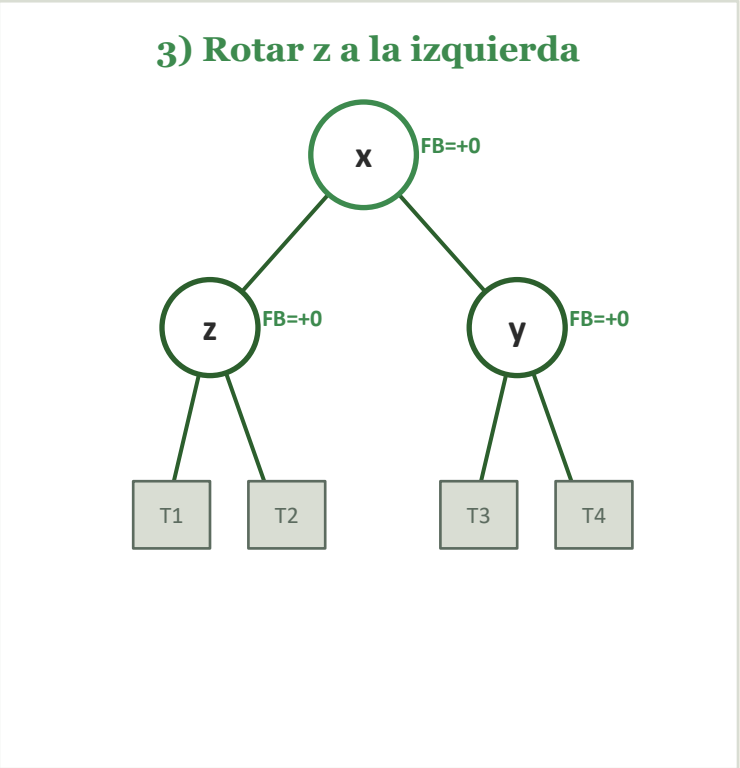
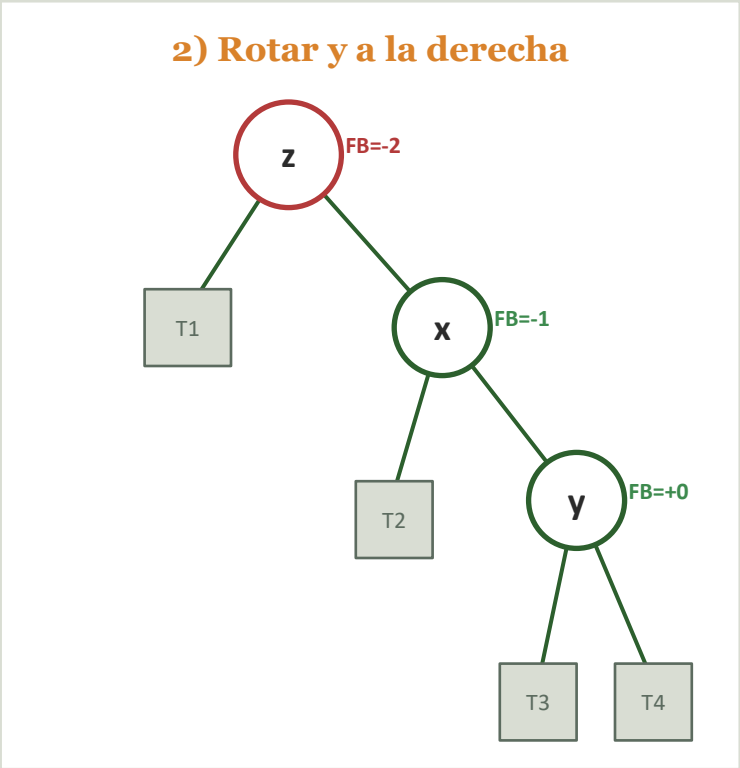
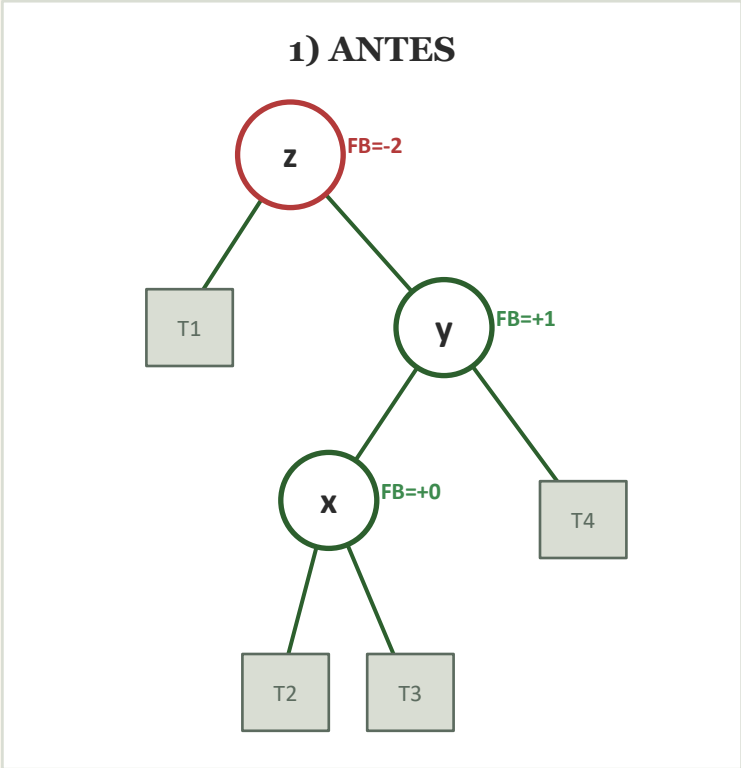


Regla: rotación doble = rotación simple en el hijo + rotación simple opuesta en el nodo desbalanceado.

Rotación doble Derecha-Izquierda (caso RL)

Se aplica una rotación derecha al hijo, después una rotación izquierda al nodo

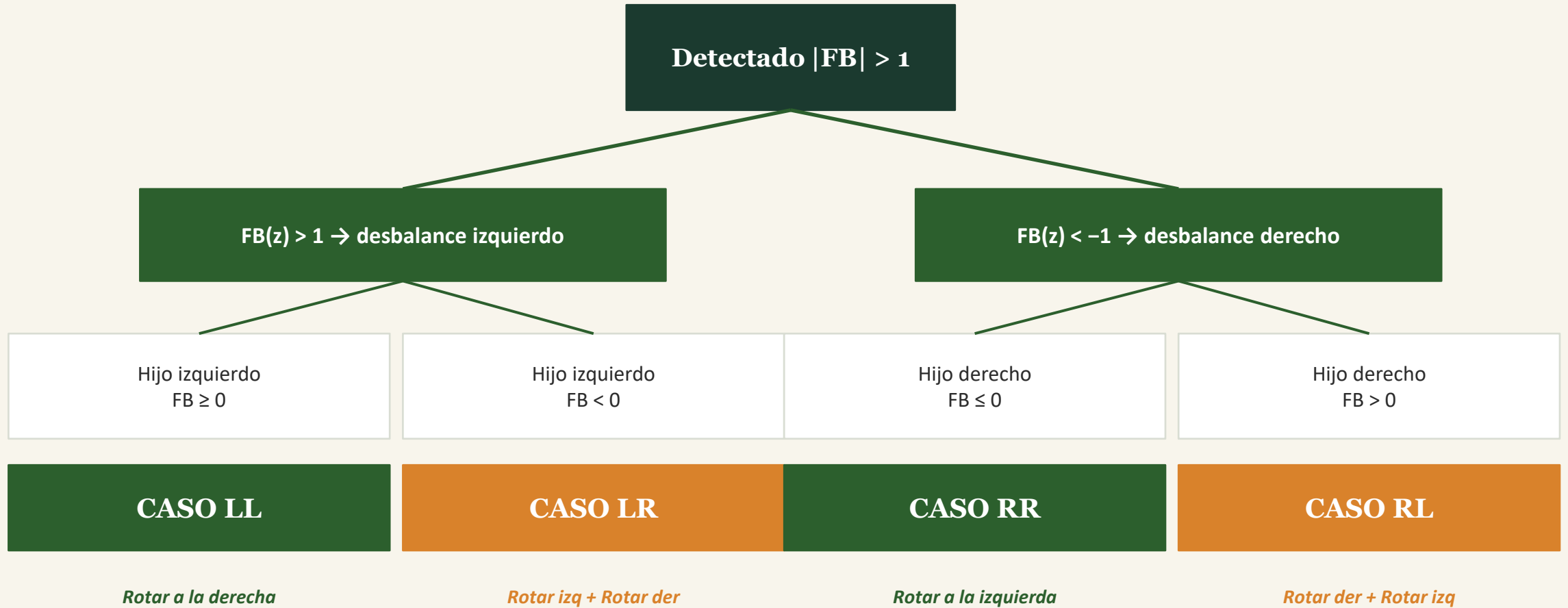
CASO RL



RL es el espejo de LR. Identifica primero el caso (LL, RR, LR, RL) y aplica las rotaciones correspondientes en orden.

¿Qué rotación aplicar?

Diagrama de decisión basado en el factor de balance

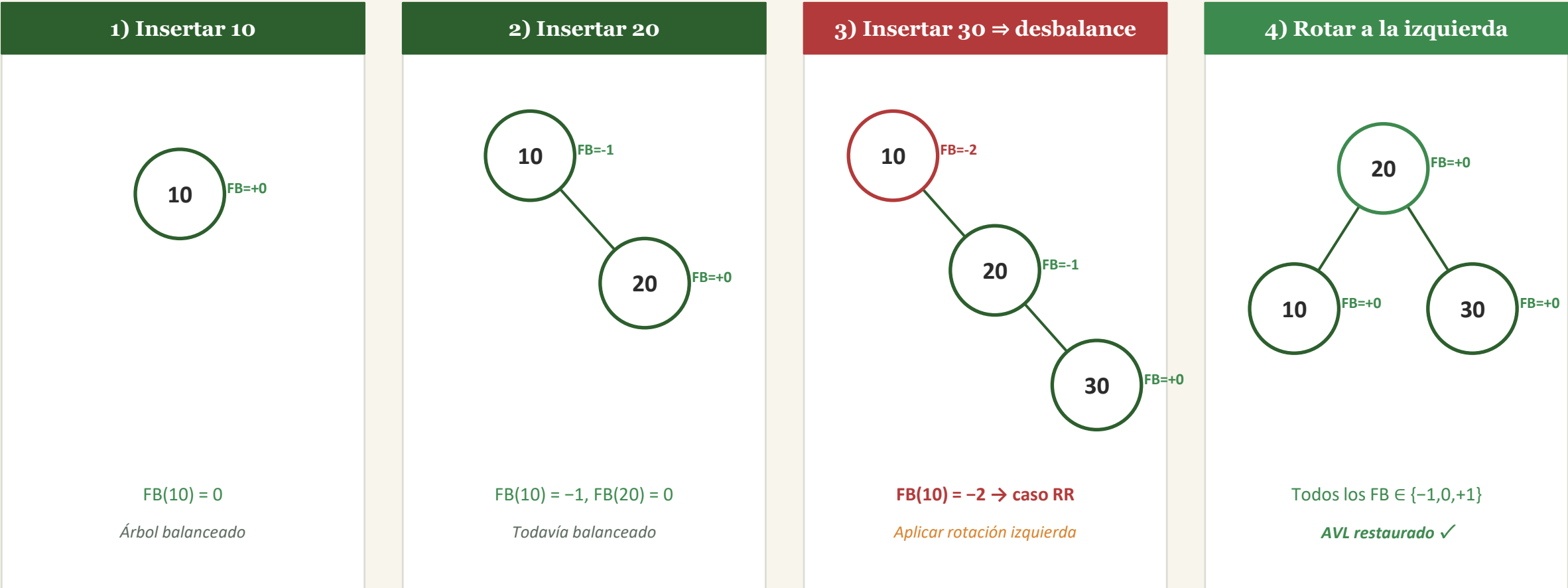


Convención: $FB = altura(izq) - altura(der)$. Si tu libro usa la convención inversa, los signos cambian, pero la lógica es la misma.

Ejemplo completo: insertar 10, 20, 30, 40, 50, 25

Parte 1 — primeras inserciones y primer rebalanceo

Insertamos los valores uno a uno y mostramos el árbol después de cada operación.

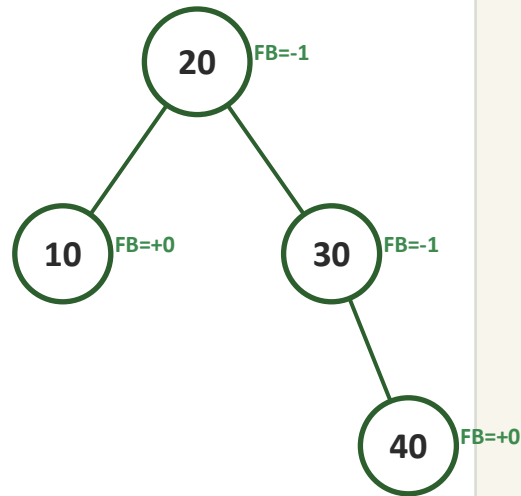


Continuamos insertando 40, 50 y 25 en la siguiente diapositiva →

Ejemplo completo (continuación)

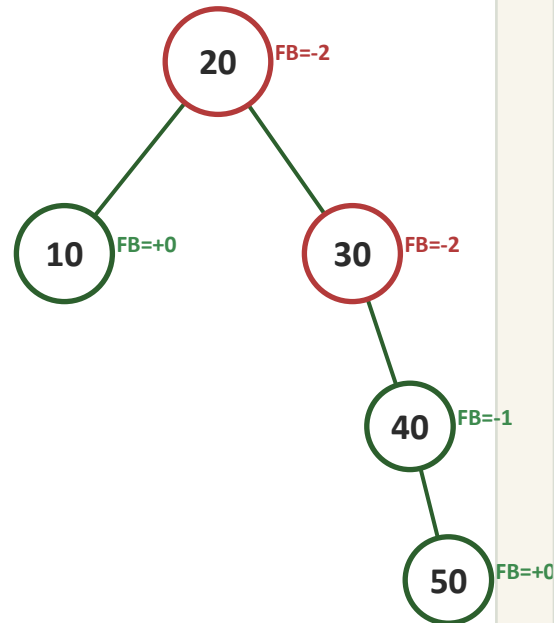
Parte 2 — segunda rotación y caso doble

5) Insertar 40



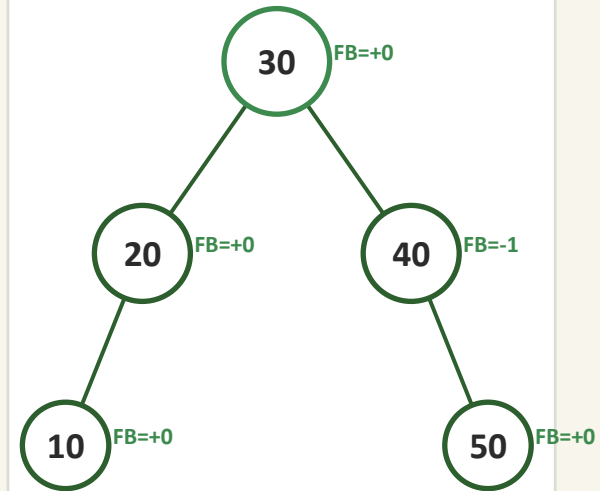
Sigue balanceado

6) Insertar 50 ⇒ desbalance



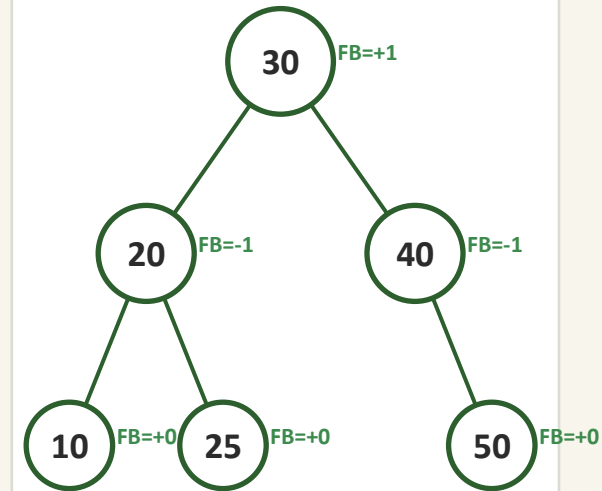
$FB(20) = -2 \rightarrow$ caso RR

7) Rotar a la izquierda



Árbol balanceado ✓

8) Insertar 25



Todos los $FB \in \{-1, 0, 1\}$



Resultado final: árbol AVL con 6 nodos, altura 2, todas las búsquedas en $O(\log n)$.

Algoritmo de inserción

Inserción estándar de BST + paso de rebalanceo

Pasos a alto nivel

1

Inserción BST

Bajar comparando hasta encontrar la posición y enlazar el nodo nuevo.

2

Actualizar altura

De abajo hacia arriba en el camino recorrido.

3

Calcular FB

$FB(v) = altura(izq) - altura(der)$ en cada ancestro.

4

Detectar $|FB| > 1$

Si lo hay, identificar el caso (LL, RR, LR o RL).

5

Rebalancear

Aplicar la rotación correspondiente y devolver la nueva raíz.

`insertar(nodo, clave)`

```
if nodo == null: return Nodo(clave)
if clave < nodo.clave:
    nodo.izq = insertar(nodo.izq, clave)
elif clave > nodo.clave:
    nodo.der = insertar(nodo.der, clave)
else: return nodo          # no duplicados

nodo.alt = 1 + max(alt(nodo.izq), alt(nodo.der))
fb = altura(nodo.izq) - altura(nodo.der)

# LL
if fb > 1 and clave < nodo.izq.clave:
    return rotarDer(nodo)
# RR
if fb < -1 and clave > nodo.der.clave:
    return rotarIzq(nodo)
# LR
if fb > 1 and clave > nodo.izq.clave:
    nodo.izq = rotarIzq(nodo.izq)
    return rotarDer(nodo)
# RL
if fb < -1 and clave < nodo.der.clave:
    nodo.der = rotarDer(nodo.der)
    return rotarIzq(nodo)
return nodo
```

Eliminación en un AVL

Igual que en un BST + rebalanceo desde el padre del nodo eliminado

Tres casos de eliminación (igual que en un BST):

1

Caso 1: Hoja

Eliminar el nodo directamente (poner su enlace a null en el padre).

2

Caso 2: Un solo hijo

Sustituir el nodo por su único hijo (saltar el enlace).

3

Caso 3: Dos hijos

Reemplazar por su predecesor o sucesor in-order y eliminar este último.



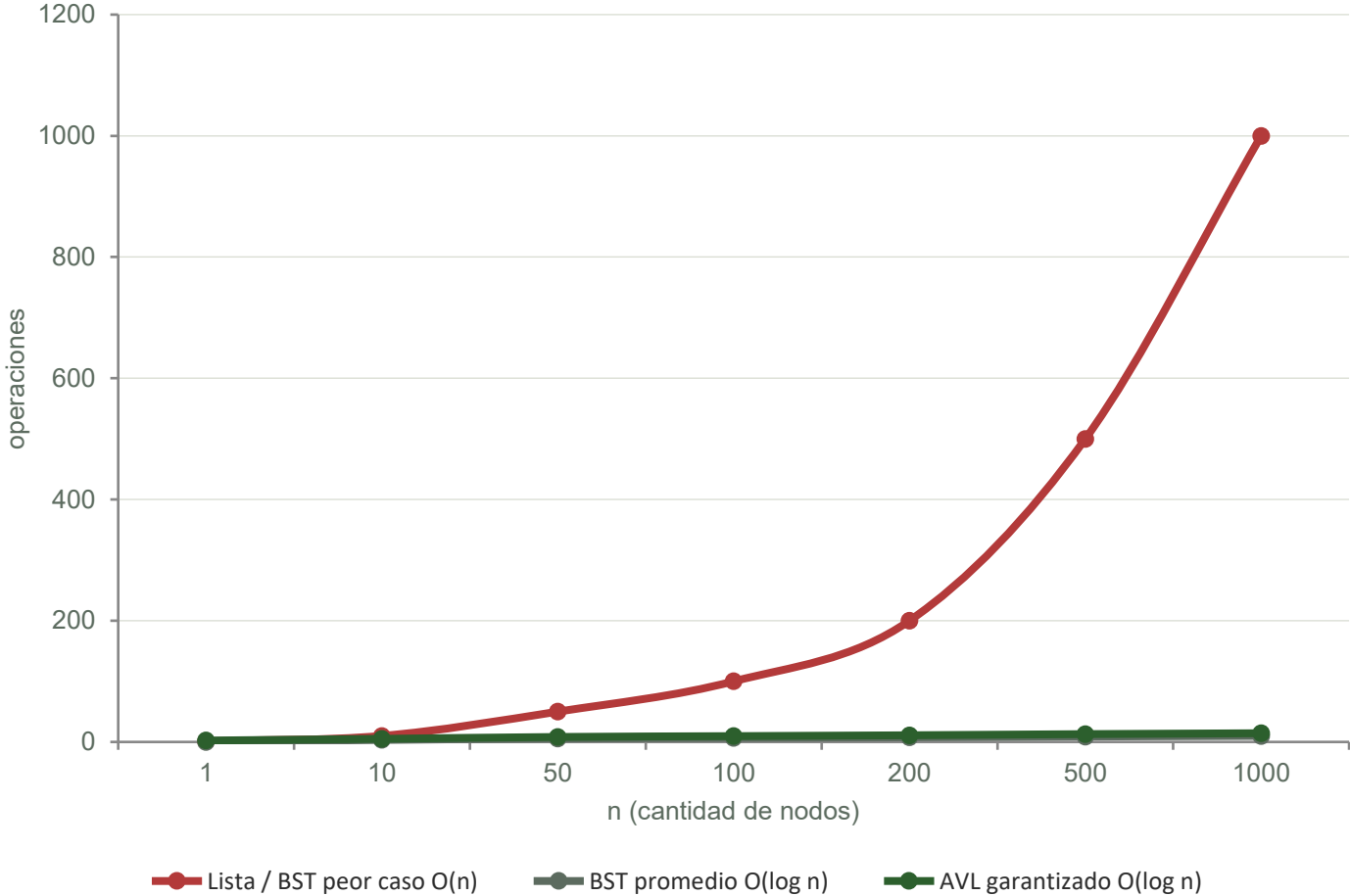
Rebalanceo posterior

1. Subir desde el padre del nodo eliminado hasta la raíz.
2. Actualizar altura y FB en cada ancestro.
3. Si $|FB| > 1$, aplicar la rotación correspondiente (LL, RR, LR, RL).
4. Diferencia clave: en eliminación pueden necesitarse **varias rotaciones en cascada** hasta la raíz; en inserción basta con una.

Complejidad computacional

AVL garantiza tiempo logarítmico en todas las operaciones

Número de comparaciones según n



Resumen

Buscar	$O(\log n)$
Insertar	$O(\log n)$
Eliminar	$O(\log n)$
Mínimo / Máximo	$O(\log n)$
In-order (recorrido)	$O(n)$
Espacio	$O(n)$

Altura máxima:

$$h \leq 1.44 \cdot \log_2(n + 2)$$

(cota probada de un AVL)

AVL vs. otros árboles balanceados

¿Cuándo conviene usar AVL?

Característica	AVL	Rojo-Negro	B-Tree
Balance	Estricto ($ FB \leq 1$)	Aproximado	Por nodos / claves
Búsqueda	Más rápida (árbol más bajo)	Ligeramente más alta	Pocos accesos a disco
Inserción / Eliminación	Más rotaciones	Menos rotaciones	División de nodos
Información extra por nodo	altura (entero)	1 bit (color)	claves múltiples
Uso típico	Datos en memoria con muchas búsquedas	STL, kernels, lenguajes	Sistemas de archivos, BD



¿Cuándo elegir AVL?

Cuando dominan las búsquedas sobre las modificaciones, los datos caben en RAM, y se requiere una garantía estricta de altura logarítmica.

Aplicaciones prácticas

Dónde encontramos árboles AVL en sistemas reales



Tablas de símbolos

Compiladores e intérpretes mantienen identificadores, tipos y ámbitos en árboles balanceados para resolver referencias rápidamente.



Índices en memoria

Bases de datos en memoria caché o motores de consulta crean índices AVL para acelerar búsquedas en columnas pequeñas y cambiantes.



Rutas de red

Algunos protocolos de enrutamiento mantienen tablas de prefijos en estructuras balanceadas tipo AVL para búsquedas $O(\log n)$.



Sistemas de ventanas

Ordenar y buscar regiones, eventos o componentes visuales con muchas inserciones y consultas a lo largo del tiempo.



Aplicaciones educativas

AVL es el ejemplo clásico para enseñar invariantes, balance y demostración de complejidad amortizada en cursos de estructura de datos.



Conjuntos / mapas ordenados

Implementaciones de TreeSet / TreeMap en algunas bibliotecas usan AVL cuando se requiere altura mínima estricta.

Ejercicios propuestos

Para practicar en clase y como tarea

1

Inserción guiada

Construye paso a paso el AVL al insertar: 50, 30, 70, 20, 40, 60, 80, 10, 25. Indica el FB de cada nodo en cada paso.

2

Detección de caso

Dado este AVL con $FB(\text{raíz}) = +2$ y $FB(\text{hijo izq}) = -1$, ¿qué caso es y qué rotaciones aplicas? Justifica.

3

Rotaciones por caso

Diseña un ejemplo mínimo de 3 nodos para cada uno de los 4 casos (LL, RR, LR, RL). Dibuja antes y después.

4

Eliminación con cascada

A partir del árbol del ejercicio 1, elimina los nodos 80, 70 y 50 en ese orden. Documenta todas las rotaciones.

5

Implementación

Implementa en Python las funciones: insertar, eliminar, altura, factorBalance y las cuatro rotaciones. Adjunta pruebas unitarias.

6

Análisis comparativo

Genera 10 000 inserciones aleatorias en un BST y en un AVL. Compara la altura final y el tiempo total. Entrega gráficas.

Consejos y errores comunes

Lo que más confunde al implementar y resolver ejercicios

Recomendaciones

- Guarda la altura como entero en cada nodo, no recalculas en cada llamada.
- Después de cada inserción/eliminación, actualiza alturas de abajo hacia arriba.
- Para decidir entre LL y LR, mira el FB del HIJO, no del nieto.
- Si dudas, dibújalo: el caso siempre se identifica con dos signos (FB padre, FB hijo).
- Implementa primero las dos rotaciones simples; las dobles son combinaciones.
- Prueba con secuencias ordenadas (1, 2, 3, ...): es el peor caso para un BST y un buen test del rebalanceo.

Errores frecuentes

- Confundir la dirección de la rotación: el nodo desbalanceado baja.
- Olvidar actualizar la altura de los dos nodos involucrados en la rotación.
- Aplicar la rotación equivocada: insertar en hijo izq y rotar a la izquierda.
- Asumir que basta una rotación al eliminar (puede haber cascada).
- Mezclar las convenciones de FB (izq – der vs. der – izq).
- Modificar las referencias del padre sin reconectar a la nueva raíz del subárbol.

Para recordar

Árboles AVL — síntesis

✓ **Un AVL es un BST**

donde $|FB(v)| \leq 1$ para todo nodo.

✓ **Inserción: 1 rotación**

La primera rotación al subir restaura el balance global.

✓ **Costo garantizado**

Buscar, insertar y eliminar son $O(\log n)$.

✓ **Cuatro casos posibles**

LL y RR se resuelven con rotación simple; LR y RL con doble.

✓ **Eliminación: posibles cascadas**

Hay que verificar el balance hasta la raíz.

✓ **Altura acotada**

$h \leq 1.44 \cdot \log_2(n + 2)$.

¿Preguntas? Practicar es la mejor forma de dominar las rotaciones.